CS3230 Algorithm Analysis

"Math for AA"

NUS National University of Singapore
School of Computing

❑ Lecture Topics and Readings

❖ Simple Algorithm Analysis	CS1020/2010
❖ Algorithm Analysis Pattern	[LHW] A&E
❖ Asymptotic Notations	[CLRS]-C3
❖ Functions & Summations (Review)	[CLRS]-C3, App

Quickly get up to speed.

Hon Wai Leong, NUS
© Leong Hon Wai, 2007--

We start with something trivial.

© Leong Hon Wai, 2007--

Trivial Algorithm Analysis

```
PASS-FAIL (Mark)   ▷                    "pseudocode"
    Print Mark
    if Mark < 40
        then print "Student Fails"
        else print "Student Passes"
```

AA. Simple. Constant time.
(*always* take the same amount of time)
Does not vary with n.

© Leong Hon Wai, 2007--

Trivial Algorithm Analysis

```
MPG ()
    Read StartMiles, EndMiles, GasUsed
    Distance ← (EndMiles – StartMiles)
    Average ← Distance / GasUsed)
    Print "Average mileage is ", Average
    if Average >= 25
        then Print " Mileage is good"
        then Print " Mileage is not good"
    Print "Bye Bye"
```

"pseudo-code"

AA. Simple. Constant time.
(Does not vary with *n*.)

© Leong Hon Wai, 2007--

Trivial Algorithm Analysis

```
TEST (A, n)        ▷ A[1 . . n]
    Print A[1]
    key ← A[2] * 2
    if A[1] > 0  then Print "first number is positive"
    Print A[n]     (* print last element of array *)
```

"pseudo-code"

AA: Simple. Constant time.
Does not vary with n.

$$T(n) \geq 1, \quad T(n) \leq 1000, \quad T(n) = \Theta(1)$$

Lower bound

Upper bound

Guaranteed running time

© Leong Hon Wai, 2007--

Now, something more interesting

© Leong Hon Wai, 2007--

Trivial Algorithm Analysis

```
ARRAY-SUM (A, N)        ▷ A[1 . . N]
    Sum-sf ← 0
    for j ← 1 to N
        do  Sum-sf ← Sum-sf + A[j]
    Sum ← Sum-sf
```

"pseudo-code"

AA: Simple. Linear time.
Directly proportional to n.

$$T(n) \geq n, \quad T(n) \leq 10{,}000n, \quad T(n) = \Theta(n)$$

Lower bound

Upper bound

Guaranteed running time

© Leong Hon Wai, 2007--

Trivial Algorithm Analysis

```
ALL-PRODUCT (A, n, M, n)    ▷ A[1..n], M[1..n; 1..n]
    (* compute all products *)
    for k ← 1 to n
        for j ← 1 to n
            do M[k, j] ← A[k] * A[j]
```

"pseudo-code"

AA: Simple. Quadratic time.
Proportional to n^2.

$$T(n) \geq n^2, \quad T(n) \leq 10{,}000n^2, \quad T(n) = \Theta(n^2)$$

Lower bound

Upper bound

Guaranteed running time

© Leong Hon Wai, 2007--

Too boring?

is it useful?

© Leong Hon Wai, 2007--

Testing * operation in a CPU

Q: How to test that the "*" operation of your CPU is correct?

A: Check exhaustively. for all a, b
Check $a * b = c$

Q: How long will it take?

A: Any guesses?

© Leong Hon Wai, 2007--

Testing * operation in a CPU

Q: How long will it take?

Assume we use a 100G-Flop CPU
can take 100B operations per sec.

a is a 32-bit number (2^{32} cases)
b is a 32-bit number (2^{32} cases)
So, $(a * b)$ there are (2^{64} cases)

Time taken = $(2^{64} / 100\text{x}10^{9})$ sec

≈ 6 years!

© Leong Hon Wai, 2007--

WolframAlpha computational knowledge engine

2^64 / 100x10^9 seconds

Examples Random

Assuming seconds of time for "seconds" | Use seconds of arc instead

http://www.wolframalpha.com/input/?i=2^64+%2F+100x10^9+seconds ←- URL

Input interpretation:

$$\frac{2^{64}}{100 \times 10^9} \text{ seconds}$$

Unit conversions: More

3.074×10^6 minutes

51 241 hours

2135 days

305 weeks

70.19 months

© Leong Hon Wai, 2007--

Testing * operation in a CPU

Q: What if we have (n lg n) algorithm?

Assume we use a 100G-Flop CPU
can take 100B operations per sec.

a is a 32-bit number (2^{32} cases)
b is a 32-bit number (2^{32} cases)

When $n = 2^{32}$, with (n lg n) algorithm

Time taken = (2^{32} * 32 / 100x10^9) sec

< 2 sec!

© Leong Hon Wai, 2007--

Moral of the story

Analysis of algorithms
help us make *predictions*.

Analysis of algorithms
help us *prepare for the worst case*.

© Leong Hon Wai, 2007--

Story of Algorithms in Action

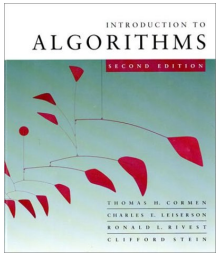
Credit card processing centre in SG (Sci Park):

- monitor showing servers load for diff. countries,
- blue, green, yellow, orange (send SMS alert),
- RED (URGENT Alert! ➔ deploy more servers!)

Note: ***Picture is NOT the real thing.***
But it gives the rough idea
and "demos" my point.

Picture is NOT very good.
Will find a better one.

© Leong Hon Wai, 2007--



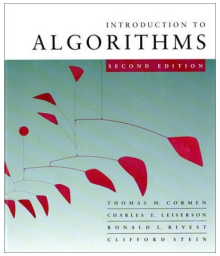
Why study algorithms and performance?

- Algorithms help us to understand ***scalability***.
- Performance often draws the line between what is feasible and what is impossible.

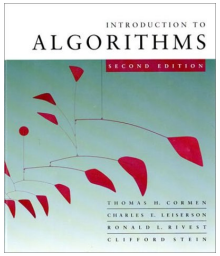
- Algorithmic mathematics provides a ***language*** for talking about program behavior.
- Performance is the ***currency*** of computing.
- The lessons of program performance generalize to other computing resources.
- Speed is fun!

Application in Web-Service

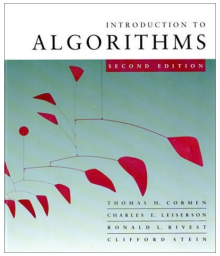
© Leong Hon Wai, 2007--



Introduction to some fun AA patterns



Now analyze some algorithms and fit into AA patterns



The problem of sorting

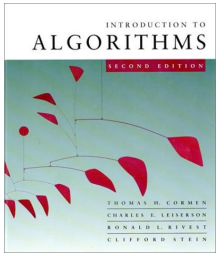
Input: sequence $\langle a_1, a_2, \ldots, a_n \rangle$ of numbers.

Output: permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ such that $a'_1 \le a'_2 \le \cdots \le a'_n$.

Example:

Input: 8 2 4 9 3 6

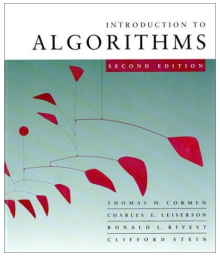
Output: 2 3 4 6 8 9



Insertion sort

"pseudocode"

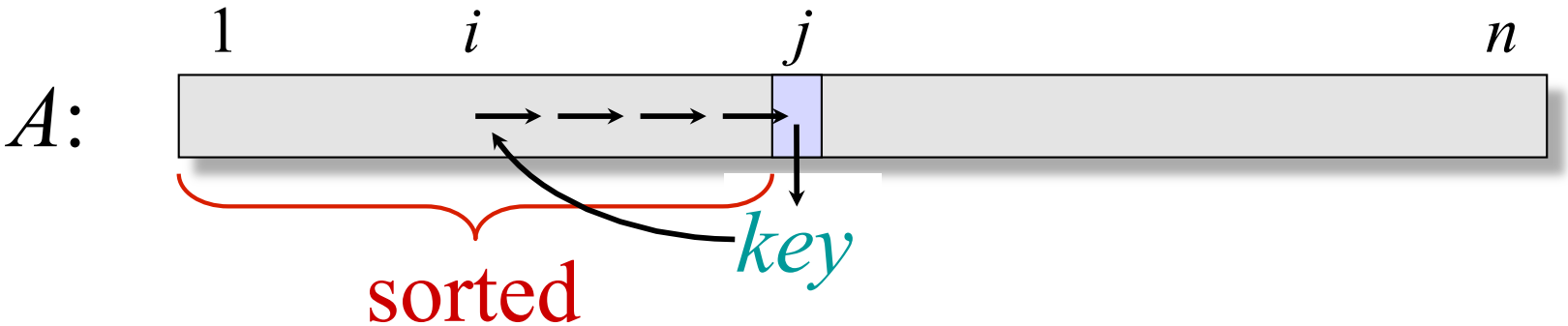
```
INSERTION-SORT (A, n)        ▷ A[1 . . n]
    for j ← 2 to n
        do key ← A[j]
           i ← j – 1
           while i > 0 and A[i] > key
               do A[i+1] ← A[i]
                  i ← i – 1
           A[i+1] = key
```

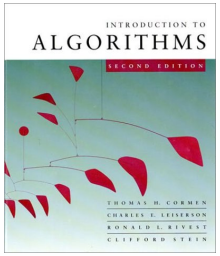


Insertion sort

"pseudocode"

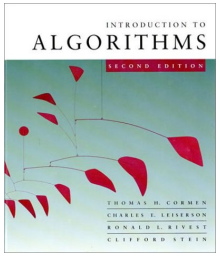
```
INSERTION-SORT (A, n)        ▷ A[1 . . n]
    for j ← 2 to n
        do key ← A[j]
           i ← j – 1
           while i > 0 and A[i] > key
               do A[i+1] ← A[i]
                  i ← i – 1
           A[i+1] = key
```





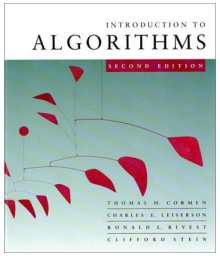
Example of insertion sort

8 2 4 9 3 6



Example of insertion sort

8 2 4 9 3 6



Example of insertion sort

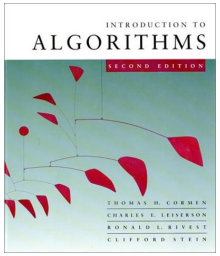
8 2 4 9 3 6

2 8 4 9 3 6

Example of insertion sort

8 2 4 9 3 6

2 8 4 9 3 6

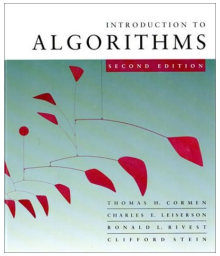


Example of insertion sort

8 2 4 9 3 6

2 8 4 9 3 6

2 4 8 9 3 6

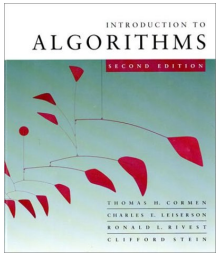


Example of insertion sort

8 2 4 9 3 6

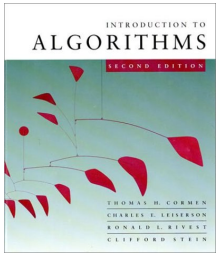
2 8 4 9 3 6

2 4 8 9 3 6



Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6



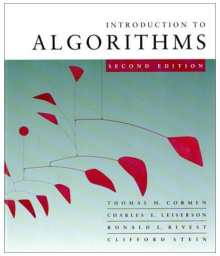
Example of insertion sort

8 2 4 9 3 6

2 8 4 9 3 6

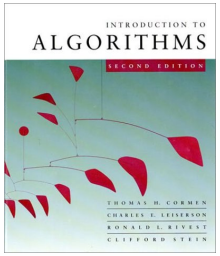
2 4 8 9 3 6

2 4 8 9 3 6



Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6
2	3	4	8	9	6



Example of insertion sort

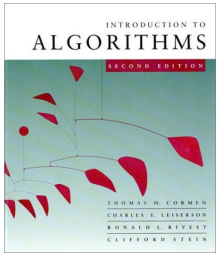
8 2 4 9 3 6

2 8 4 9 3 6

2 4 8 9 3 6

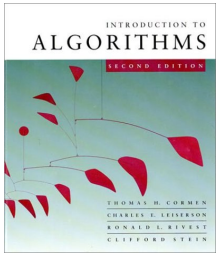
2 4 8 9 3 6

2 3 4 8 9 6



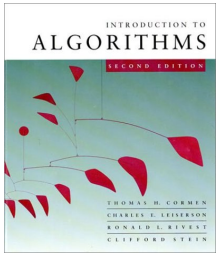
Example of insertion sort

8	2	4	9	3	6	
2	8	4	9	3	6	
2	4	8	9	3	6	
2	4	8	9	3	6	
2	3	4	8	9	6	
2	3	4	6	8	9	*done*



Running time

- The running time depends on the input: an already sorted sequence is easier to sort.
- Parameterize the running time by the size of the input, since short sequences are easier to sort than long ones.
- Generally, we seek upper bounds on the running time, because everybody likes a guarantee.



Kinds of analyses

Worst-case: (usually)

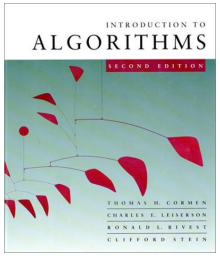
- $T(n)$ = maximum time of algorithm on any input of size n.

Average-case: (sometimes)

- $T(n)$ = expected time of algorithm over all inputs of size n.
- Need assumption of statistical distribution of inputs.

Best-case: (bogus)

- Cheat with a slow algorithm that works fast on *some* input.



Machine-independent time

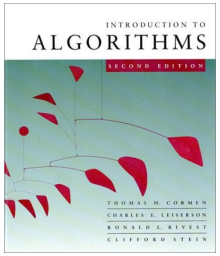
What is insertion sort's worst-case time?

- It depends on the speed of our computer:
  - relative speed (on the same machine),
  - absolute speed (on different machines).

BIG IDEA:

- Ignore machine-dependent constants.
- Look at ***growth*** of $T(n)$ as $n \to \infty$.

"Asymptotic Analysis"



Θ-notation

Math:

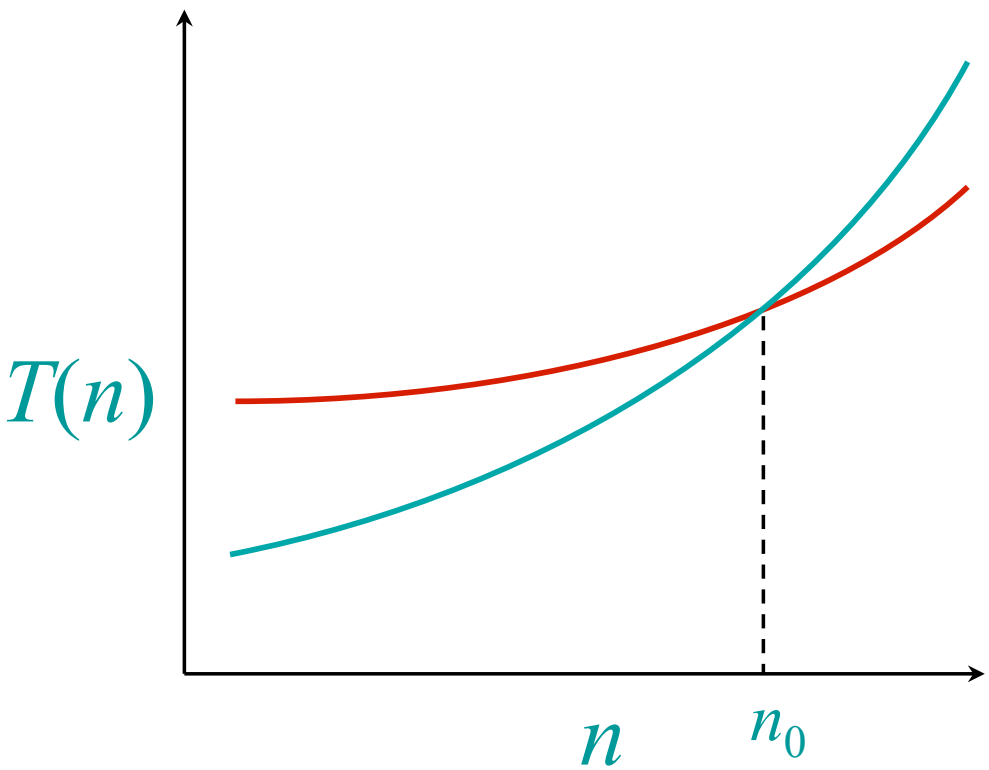
$\Theta(g(n)) = \{ f(n) :$ there exist positive constants c_1, c_2, and n_0 such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0 \}$

Engineering:

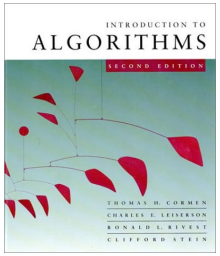
- Drop low-order terms; ignore leading constants.
- Example: $3n^3 + 90n^2 - 5n + 6046 = \Theta(n^3)$

Asymptotic performance

When n gets large enough, a $\Theta(n^2)$ algorithm *always* beats a $\Theta(n^3)$ algorithm.



- We shouldn't ignore asymptotically slower algorithms, however.
- Real-world design situations often call for a careful balancing of engineering objectives.
- Asymptotic analysis is a useful tool to help to structure our thinking.

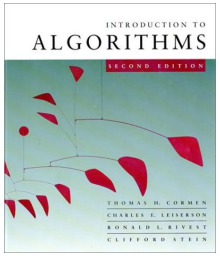


Comparing $100n^2$ vs $0.01n^3$

n	$100n^2$	$0.01n^3$	Diff	Ratio
1	100	0.01	-99.99	10000
10	10000	10	-9990	1000
100	1000000	10000	-990000	100
1000	100000000	10000000	-90000000	10
10000	10000000000	10000000000	0	1
10001	10002000100	10003000300	1000200.01	0.9999
50000	2.5E+11	1.25E+12	1E+12	0.2000

Higher order terms dominates
(ignore lower order terms)

Can ignore
constant factors



Insertion sort analysis

Worst case: Input reverse sorted.

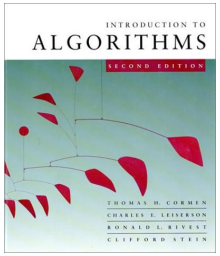
$$T(n) = \sum_{j=2}^{n} \Theta(j) = \Theta(n^2) \qquad \text{[arithmetic series]}$$

Average case: All permutations equally likely.

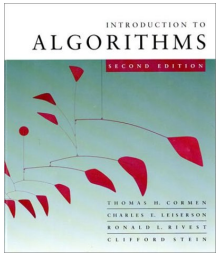
$$T(n) = \sum_{j=2}^{n} \Theta(j/2) = \Theta(n^2)$$

Is insertion sort a fast sorting algorithm?

- Moderately so, for small n.
- Not at all, for large n.



What's the AA pattern for Insertion Sort?



More sorting algorithm: Selection Sort

Sorting: Problem and Algorithms

- **Problem: Sorting**
  - Take a list of n numbers and rearrange them in increasing order
- **Algorithms:**
  - Selection Sort $\Theta(n^2)$
  - Insertion Sort $\Theta(n^2)$
  - Bubble Sort $\Theta(n^2)$
  - Merge Sort $\Theta(n \lg n)$
  - Quicksort $\Theta(n \lg n)$**

Some slides from UIT2201

Not covered in the course UIT2201

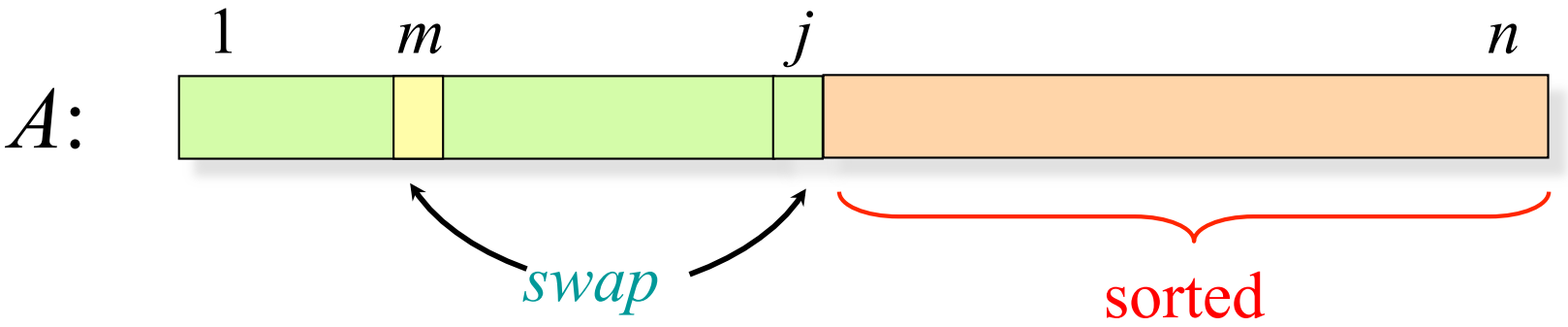
** *average case*

© Leong Hon Wai, 2003-2008

Selection sort

- **Idea behind the algorithm…**
  - **Repeatedly**
    - *find the largest number in unsorted section*
    - *Swap it to the end (the sorted section)*
  - **Re-uses the Find-Max algorithm**

$A[m]$ is largest among $A[1..j]$

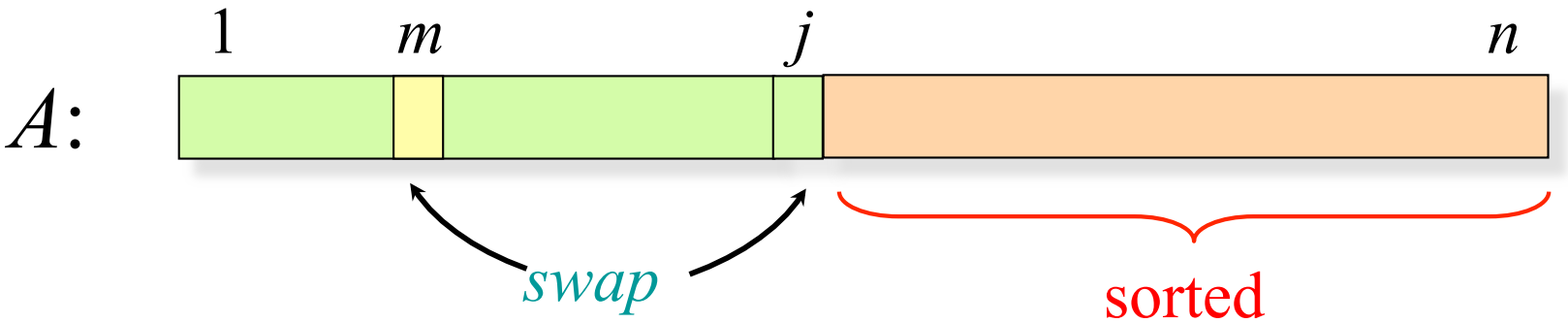


© Leong Hon Wai, 2003-2008

Selection Sort Algorithm (pseudo-code)

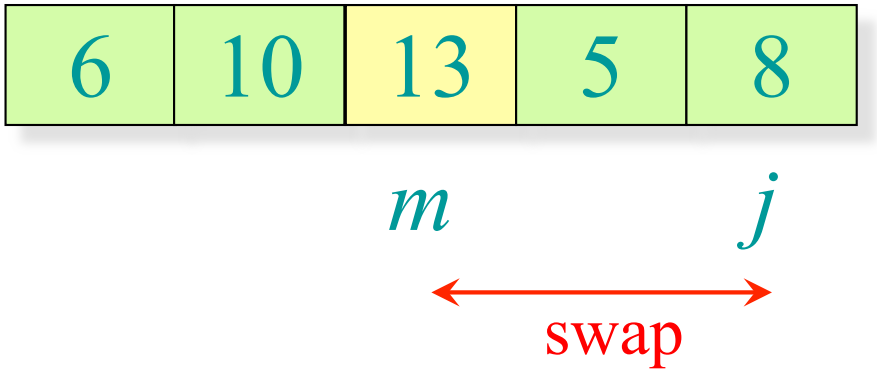
```
Selection-Sort(A, n);
begin
  j ← n;
  while (j > 1) do
    m ← Find-Max(A,j);
    swap(A[m],A[j]);
    j ← j - 1;
  endwhile
end;
```

$A[m]$ is largest among $A[1..j]$



© Leong Hon Wai, 2003-2008

Example of selection sort



© Leong Hon Wai, 2003-2008

Example of selection sort

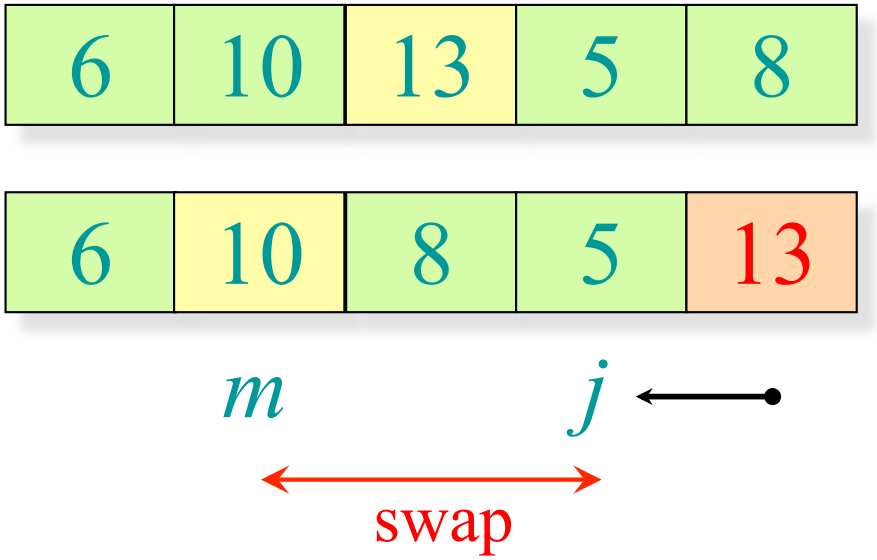
6	10	13	5	8

6	10	8	5	13

j ←

© Leong Hon Wai, 2003-2008

Example of selection sort



Example of selection sort

6	10	13	5	8

6	10	8	5	13

6	5	8	10	13

$j \longleftarrow$

© Leong Hon Wai, 2003-2008

Example of selection sort

6	10	13	5	8

6	10	8	5	13

6	5	8	10	13

j ←

m

swap

© Leong Hon Wai, 2003-2008

Example of selection sort

6	10	13	5	8

6	10	8	5	13

6	5	8	10	13

6	5	8	10	13

$j \longleftarrow$

© Leong Hon Wai, 2003-2008

Example of selection sort

6	10	13	5	8

6	10	8	5	13

6	5	8	10	13

6	5	8	10	13

m j

swap

© Leong Hon Wai, 2003-2008

Example of selection sort

6	10	13	5	8

6	10	8	5	13

6	5	8	10	13

6	5	8	10	13

5	6	8	10	13

Done. j ←

© Leong Hon Wai, 2003-2008

What about the time complexity?

- **Dominant operation: *comparisons***

6	10	13	5	8	4 comparisons
6	10	8	5	13	3 comparisons
6	5	8	10	13	2 comparisons
6	5	8	10	13	1 comparisons
5	6	8	10	13	

Done. j ←

© Leong Hon Wai, 2003-2008

Analysis of Selection Sort

Find-Max for *j* numbers takes (*j*-1) comparisons

- **When sorting *n* numbers,**
  - (n–1) comparisons in iteration 1 (when $j = n$)
  - (n–2) comparisons in iteration 2 (when $j = n$–1)
  - (n–3) comparisons in iteration 3 (when $j = n$–2)
  - . . . .
  - 2 comparisons in iteration (n-2) (when $j = 3$)
  - 1 comparisons in iteration (n-1) (when $j = 2$)

- **Total number of comparisons:**
  - Cost = $(n–1) + (n–2) + \ldots + 2 + 1 = n(n–1)/2$
    $= \Theta(n^2)$

© Leong Hon Wai, 2003-2008

Analysis of Selection Sort: Summary

- **Time complexity:** $\Theta(n^2)$
  - **Comparisons:** $n(n-1)/2$
  - **Exchanges:** n (swapping largest into place)
  - **Overall time complexity:** $\Theta(n^2)$
- **Space complexity:** $\Theta(n)$
  - $\Theta(n)$ – **space for input sequence, plus a few variables.**

Selection Sort:

Time complexity: $T(n) = \Theta(n^2)$

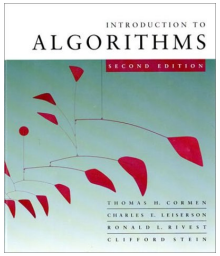
Space complexity: $S(n) = \Theta(n)$

© Leong Hon Wai, 2003-2008

AA Pattern

What is the AA Pattern for Selection Sort?

© Leong Hon Wai, 2003-2008



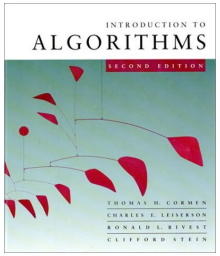
Merge sort

MERGE-SORT $A[1 \ldots n]$

1. If $n = 1$, done.
2. Recursively sort $A[\,1 \ldots \lceil n/2 \rceil\,]$ and $A[\,\lceil n/2 \rceil + 1 \ldots n\,]$.
3. "***Merge***" the 2 sorted lists.

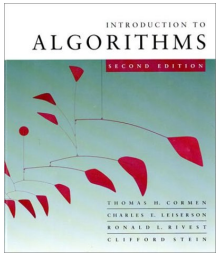
Key subroutine: **MERGE**

Mergesort is $\Theta(n \lg n)$
(shown later, also covered in CS2010/CS2020)

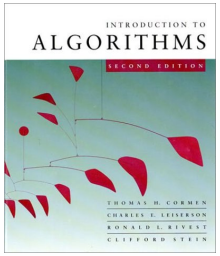


Observations about Sorting

- $\Theta(n \lg n)$ grows more slowly than $\Theta(n^2)$.
- Therefore, merge sort asymptotically beats insertion sort in the worst case.
- In practice, merge sort beats insertion sort for $n > 30$ or so.
- Go test it out for yourself!



Graph Pattern?



Unweighted graphs

Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.
Can use BFS to find shortest paths.

- Use a simple FIFO queue.

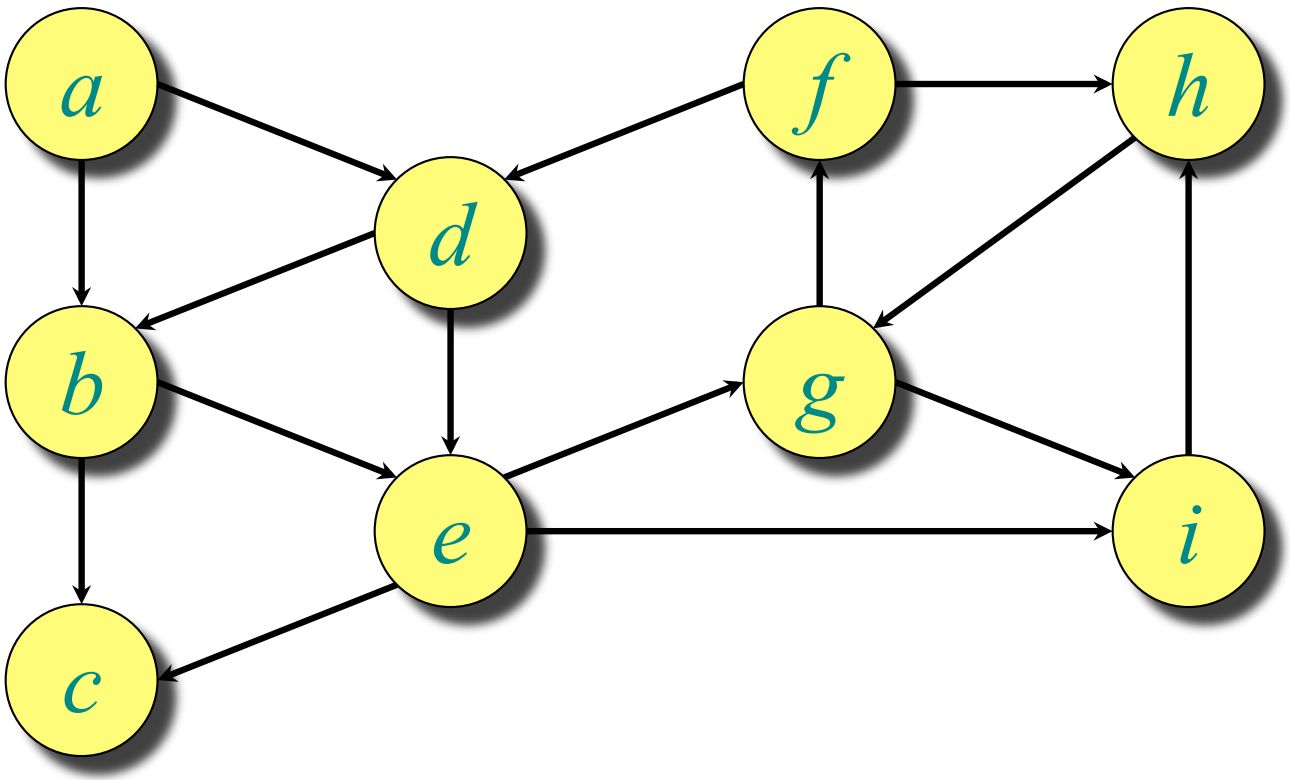
Breadth-first search

while $Q \neq \varnothing$
  do $u \leftarrow$ DEQUEUE(Q)
    for each $v \in Adj[u]$
      do if $d[v] = \infty$
        then $d[v] \leftarrow d[u] + 1$
          ENQUEUE(Q, v)

Analysis: Time $= O(V + E)$.

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

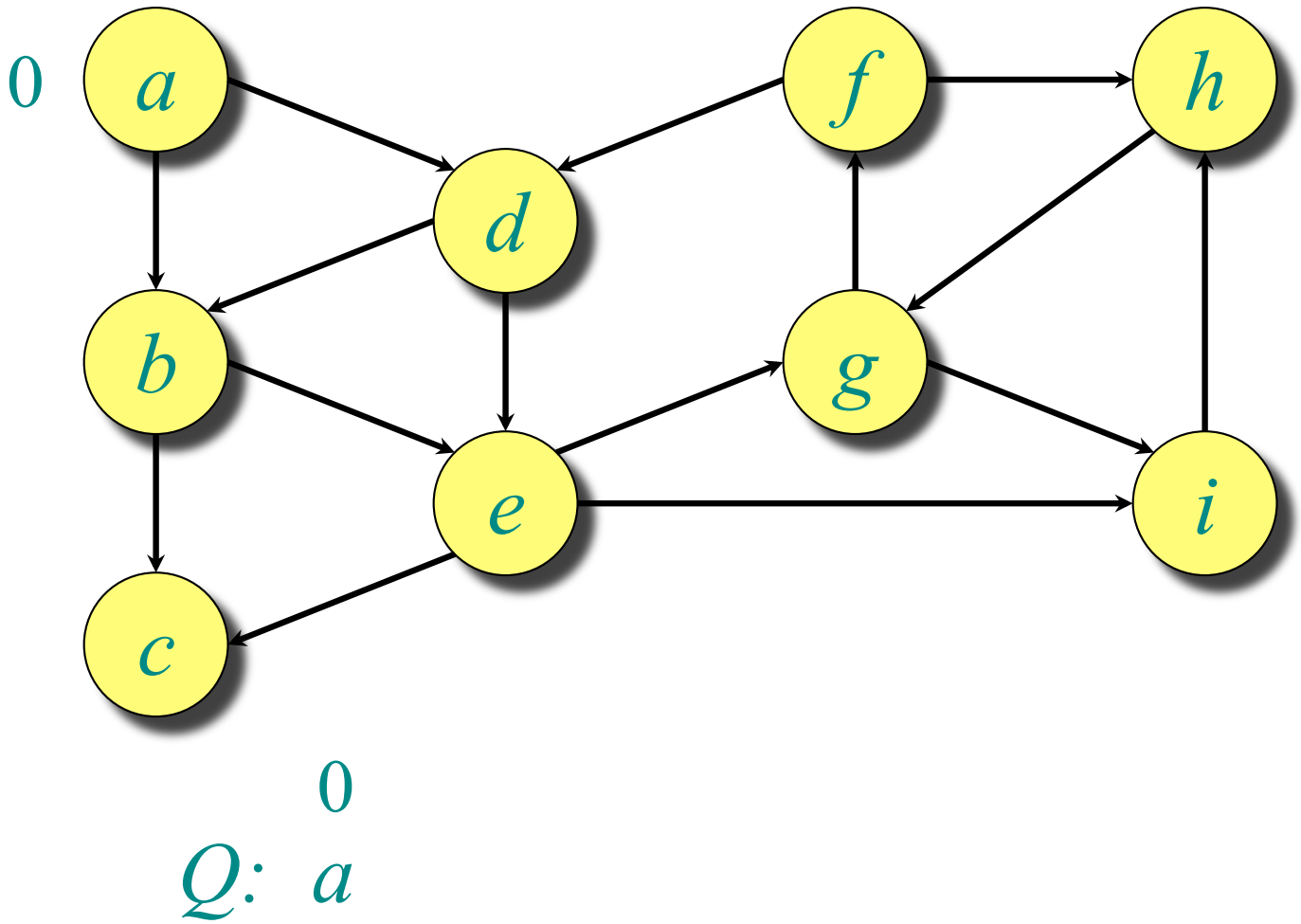
Example of breadth-first search



Q:

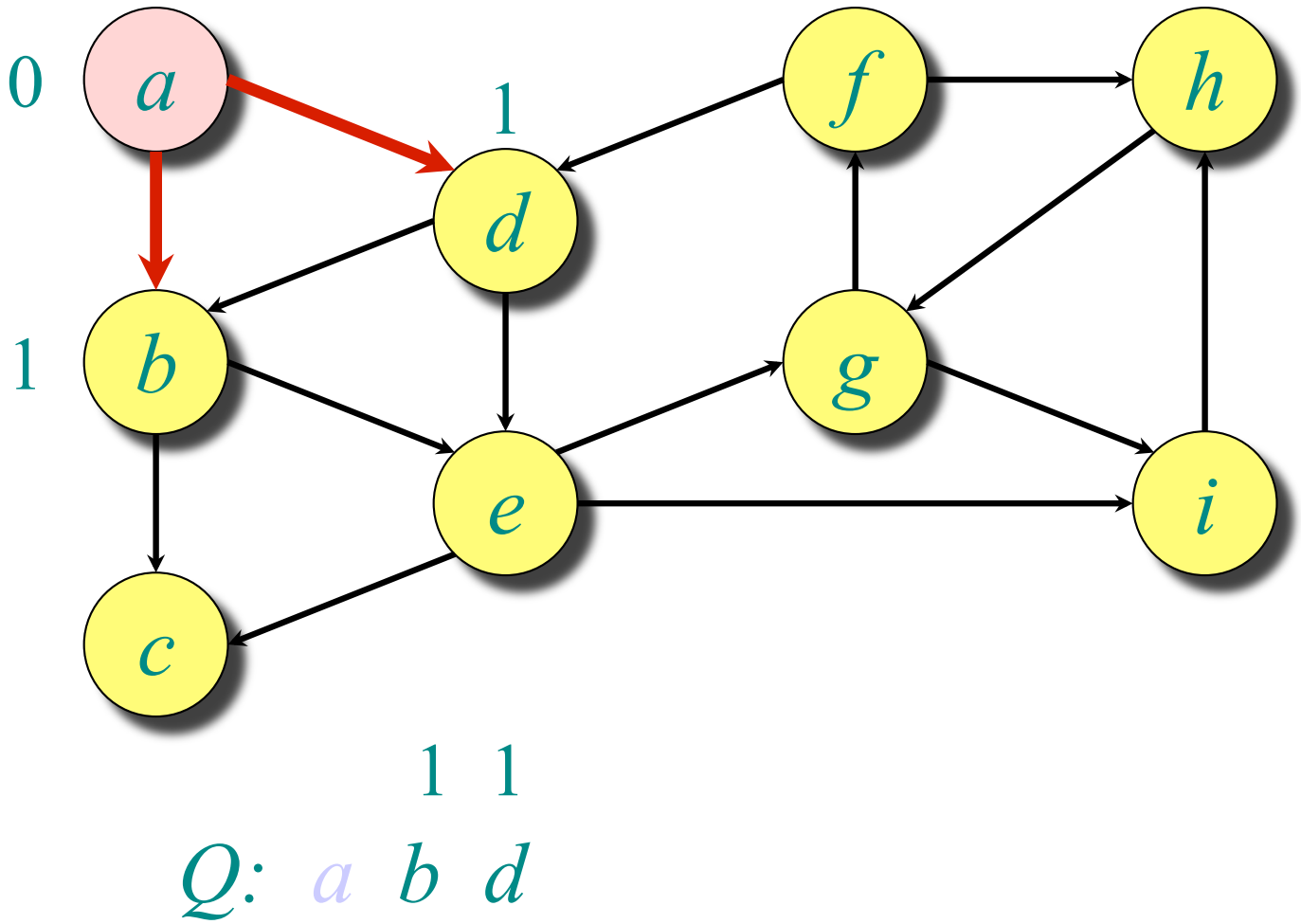
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search



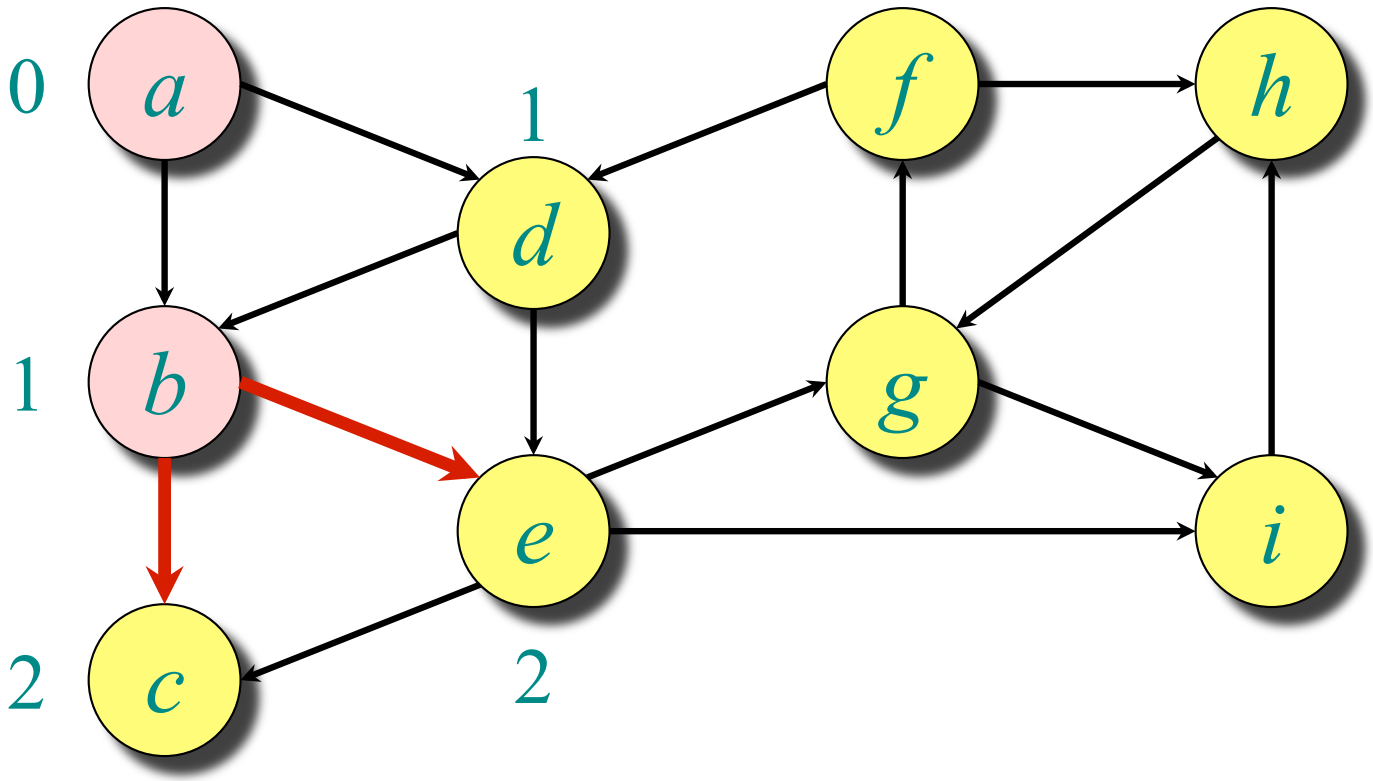
 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search



 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

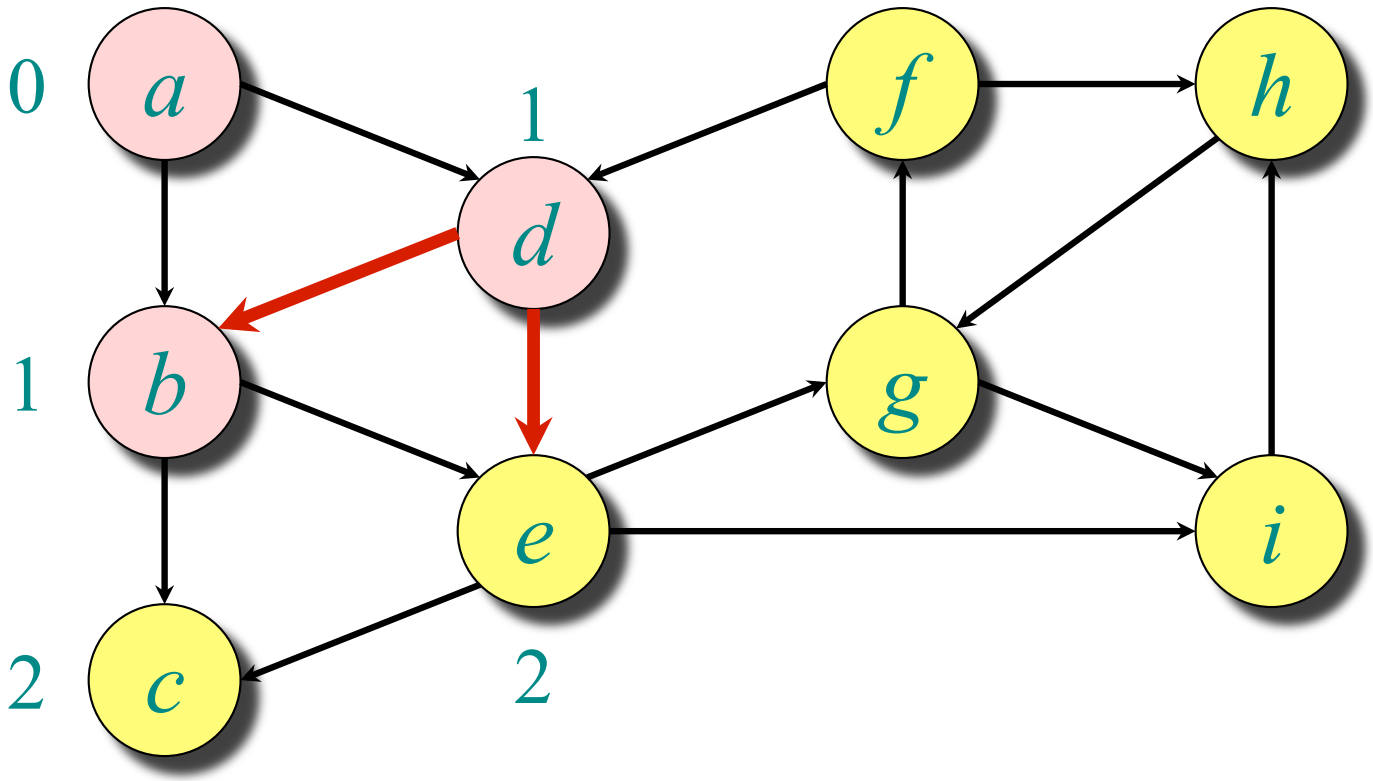


1 2 2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

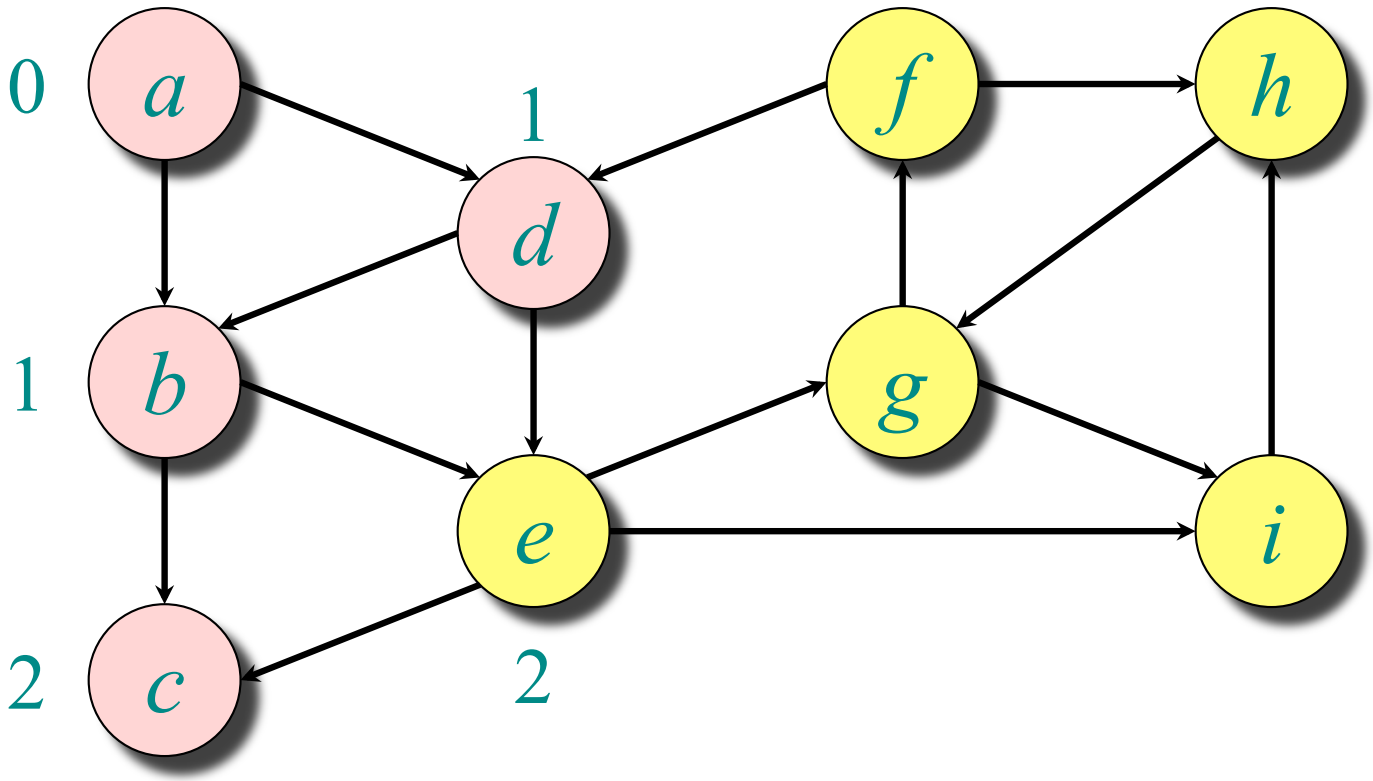


2 2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

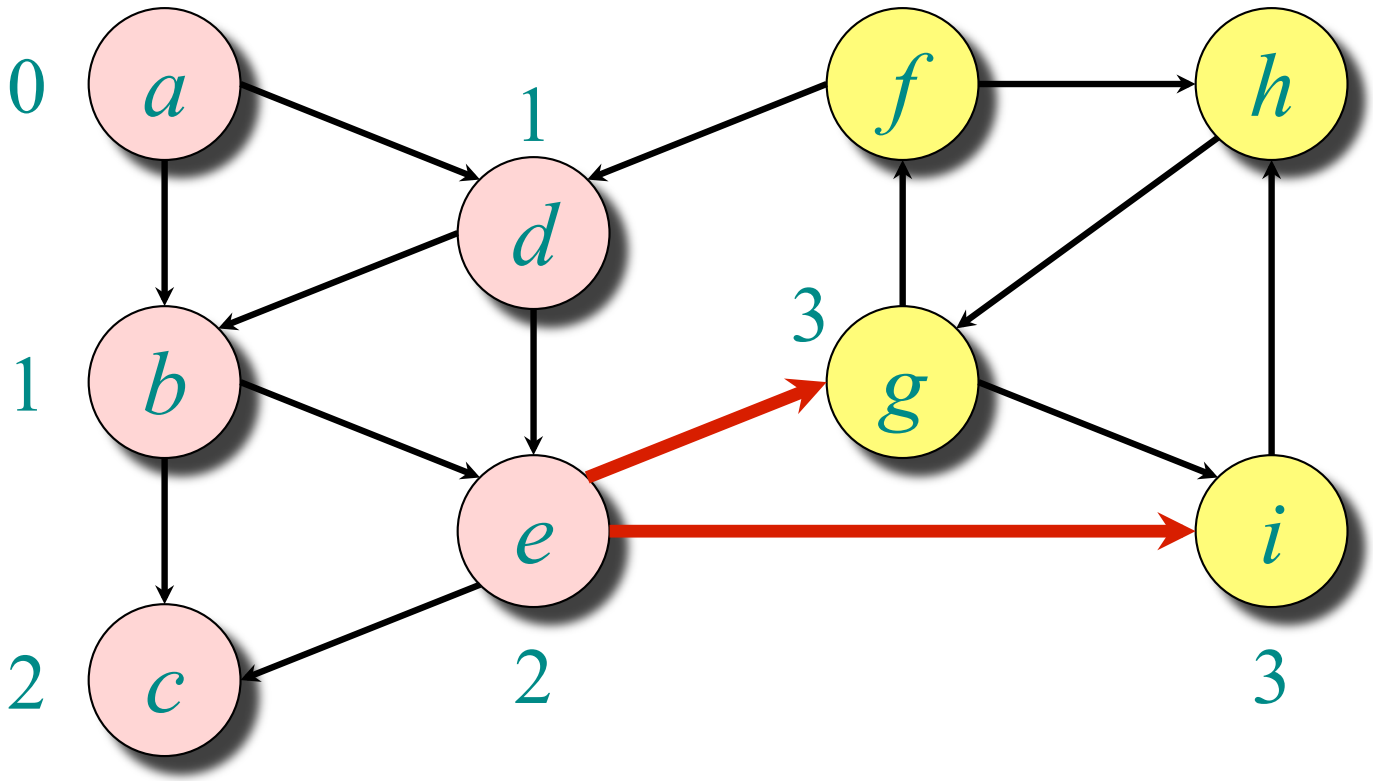


2

Q: *a* *b* *d* *c* *e*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

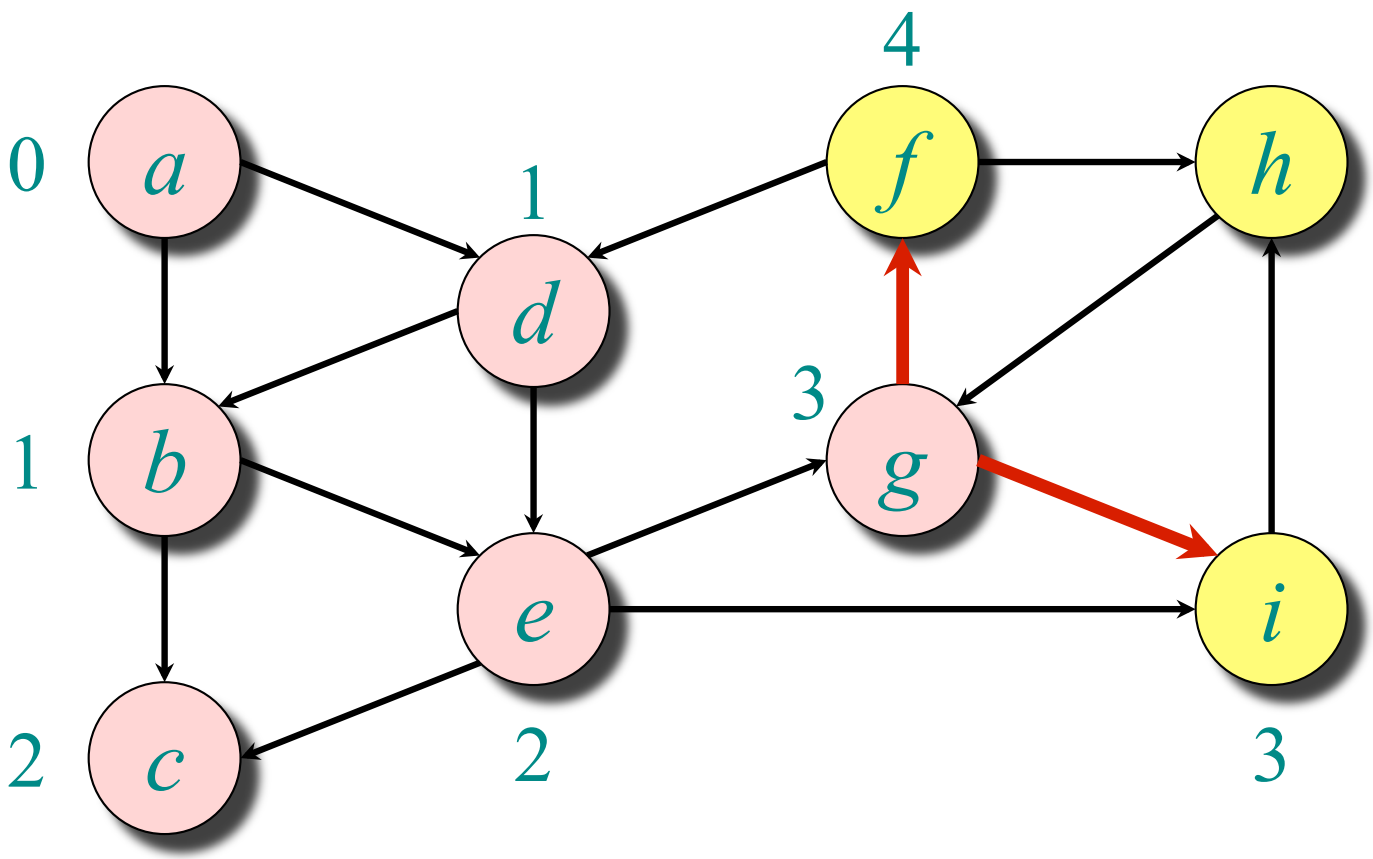


3 3

Q: *a* *b* *d* *c* *e* *g* *i*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

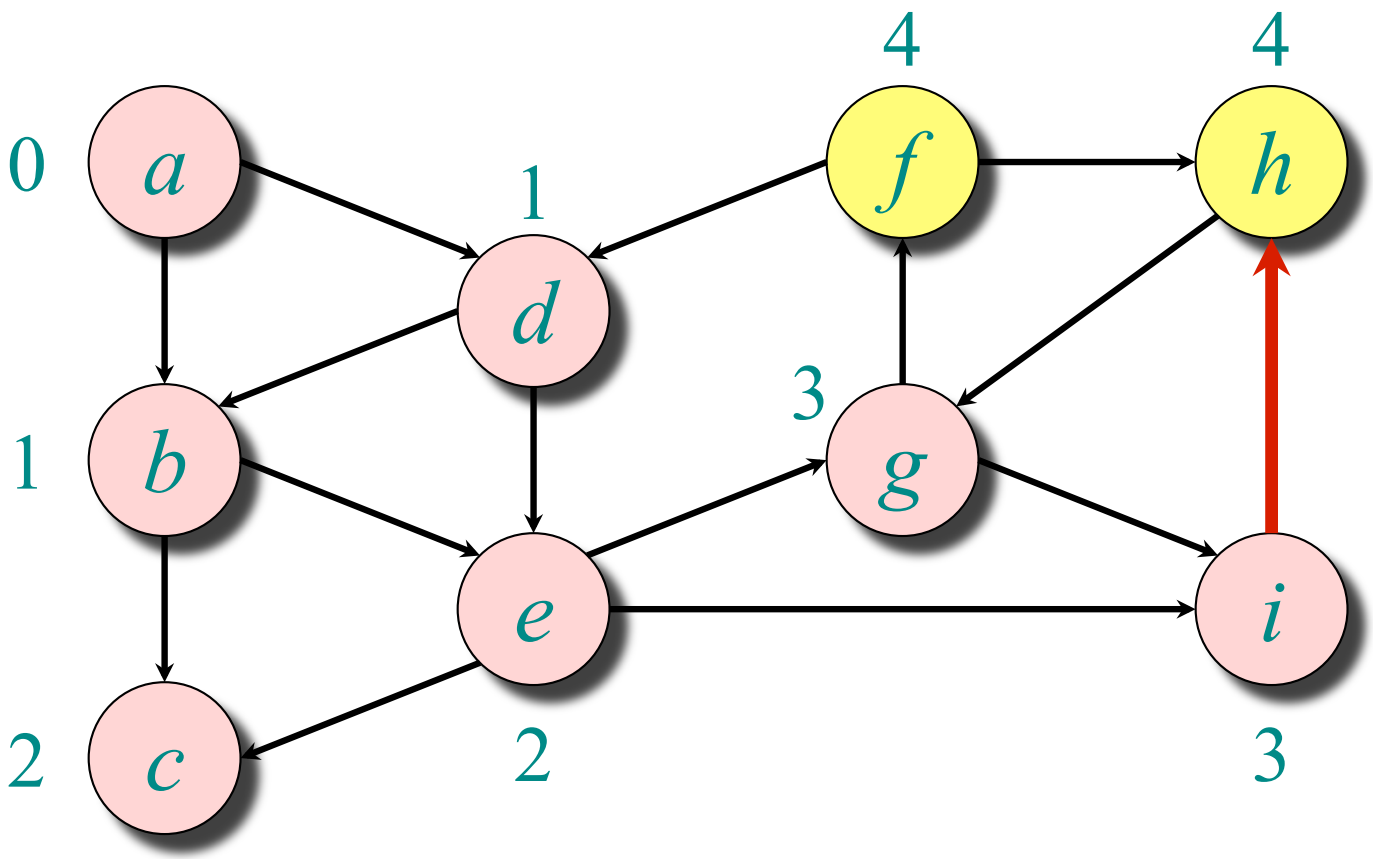


3 4

Q: *a* *b* *d* *c* *e* *g* *i* *f*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

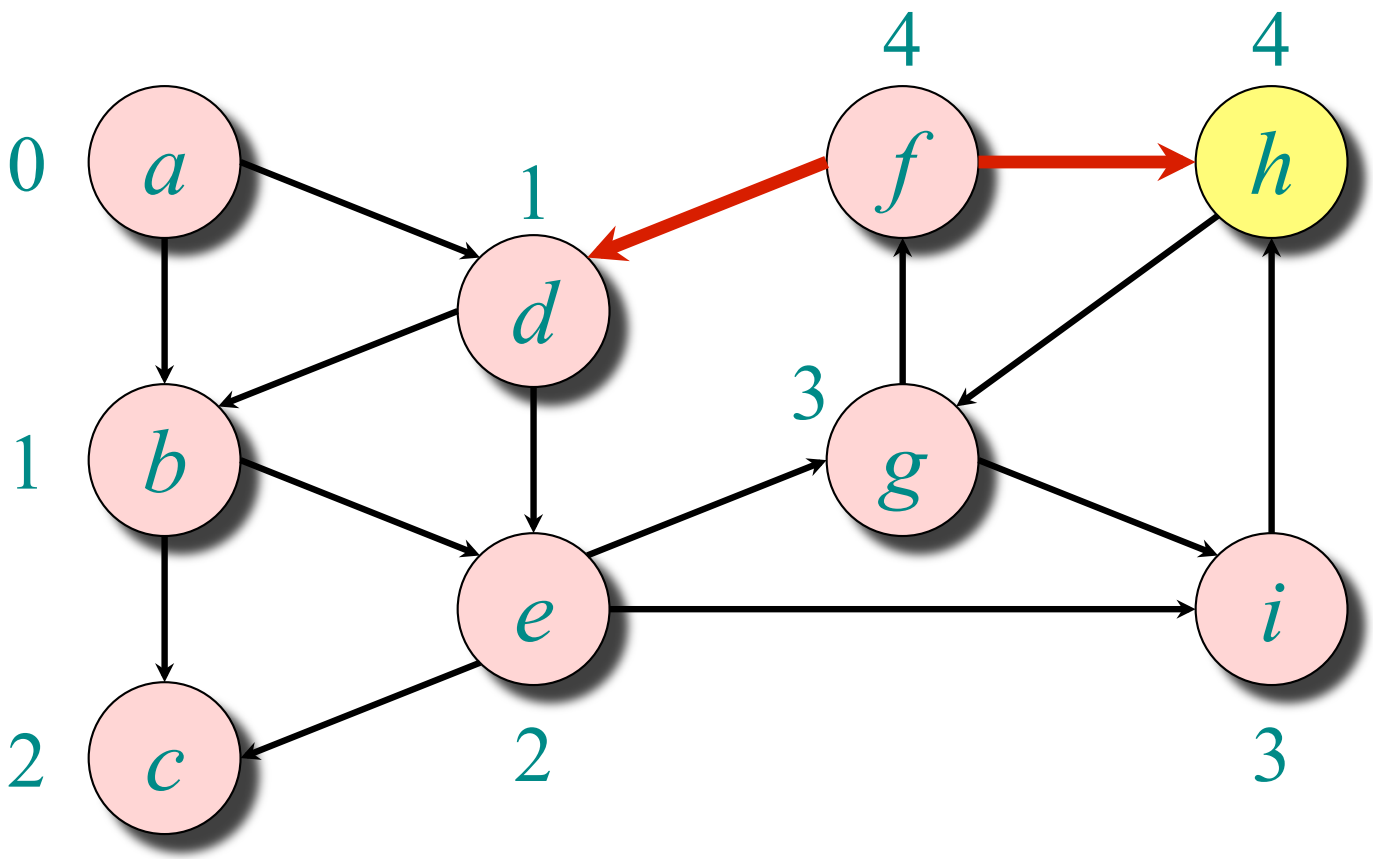


4 4

Q: *a* *b* *d* *c* *e* *g* *i* *f* *h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search

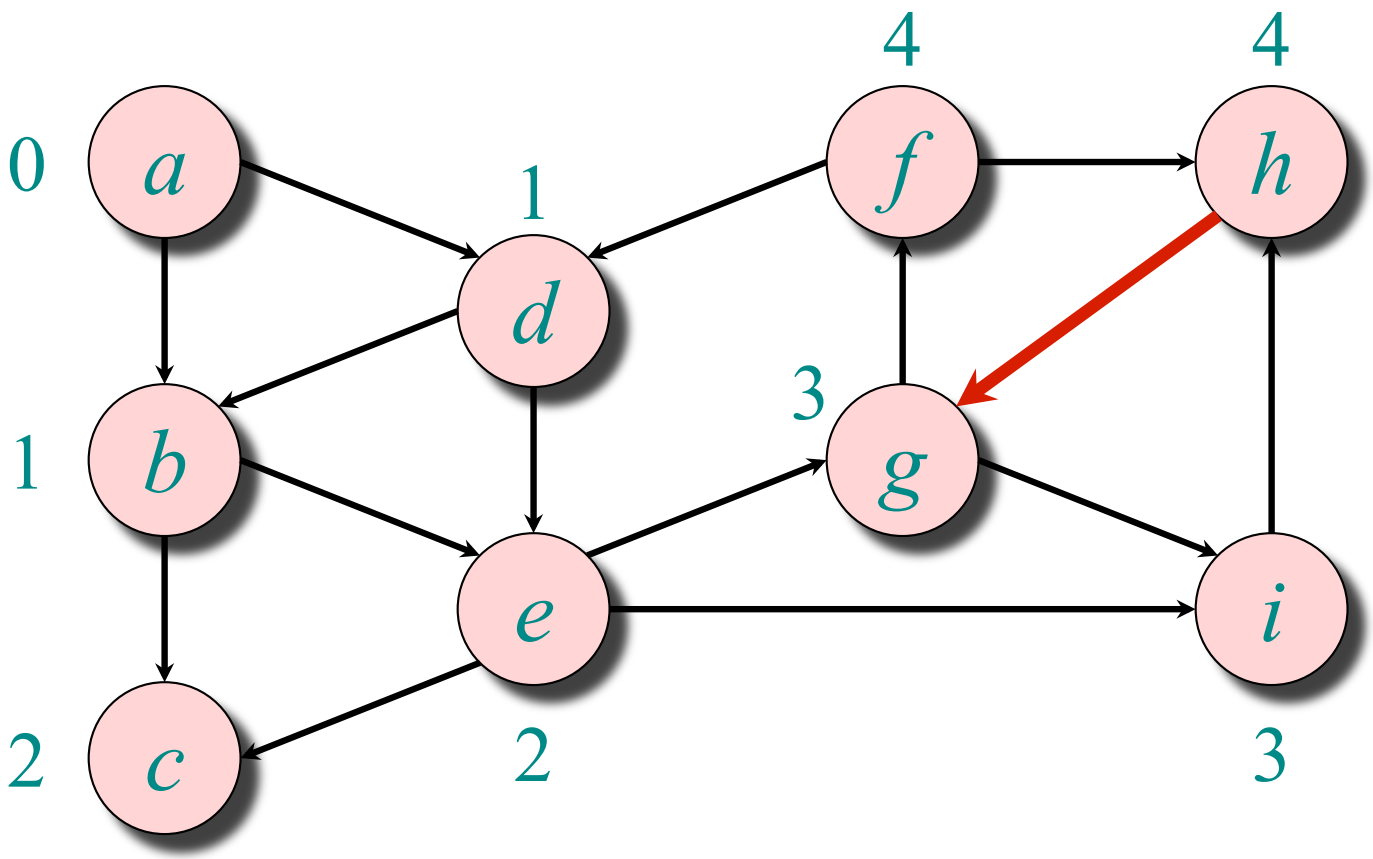


4

Q: *a b d c e g i f h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

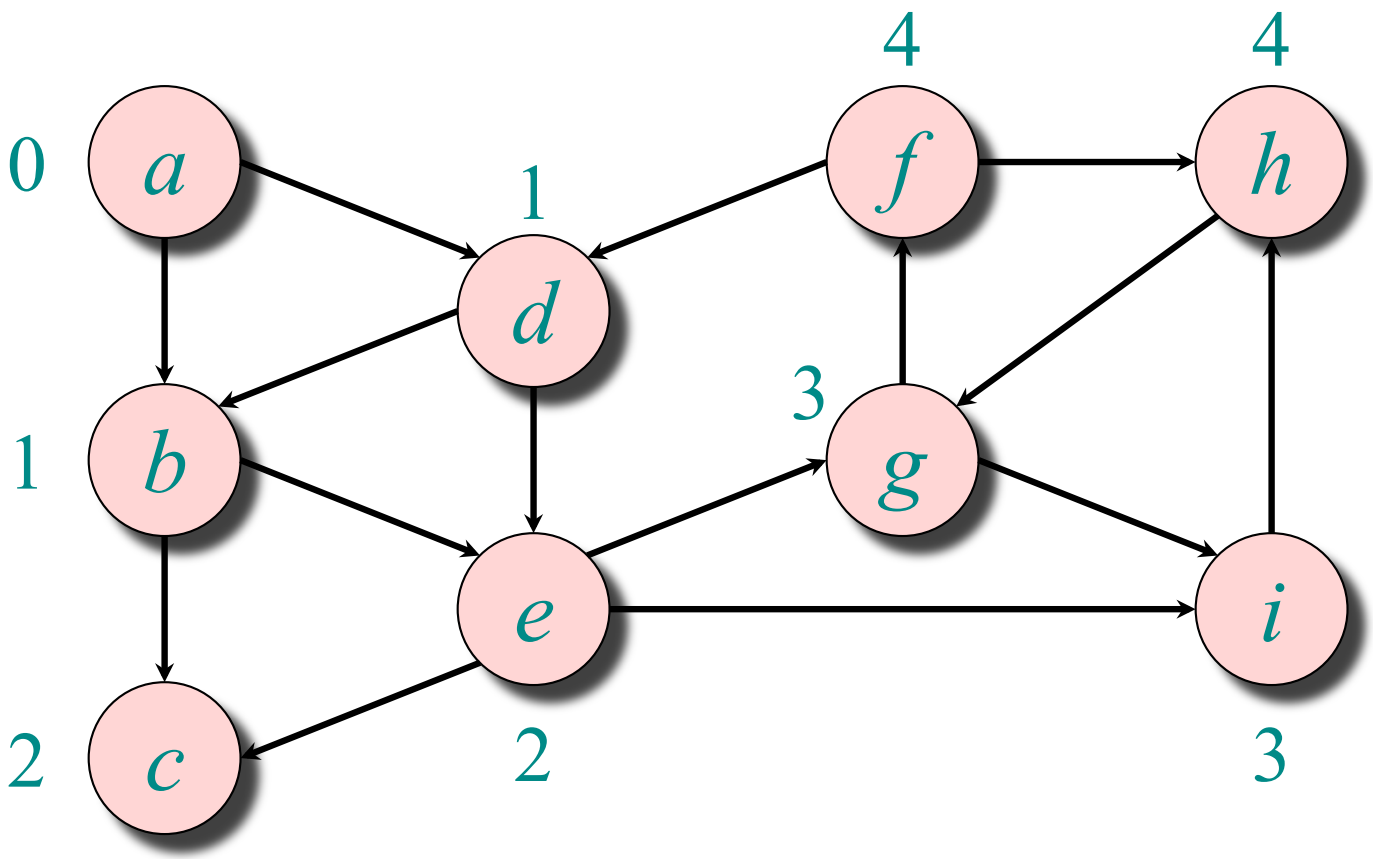
Example of breadth-first search



Q: *a b d c e g i f h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Example of breadth-first search



Q: *a b d c e g i f h*

 Copyright © 2001-5 by Erik D. Demaine and Charles E. Leiserson

Thank you.

Q & A

© Leong Hon Wai, 2003--